

Ch 3 Data Modeling Using the Entity–Relationship (ER) Model

Conceptual Modeling

- Conceptual modeling is the first and one of the most important steps in database design.
- It helps designers understand **what data should be stored, how the data is related, and what business rules should be followed** before creating the actual database.
- It focuses on **understanding the real-world problem before implementing the database.**

Database Application

- A database application consists of a database and the programs that allow users to insert, update, delete, and retrieve data.
- For example, a bank database application stores customer information and provides programs for deposits, withdrawals, and balance inquiries.

Graphical User Interface (GUI)

Application programs usually provide a **Graphical User Interface (GUI)**, which allows users to interact with the database using forms, menus, buttons, and mobile applications **instead of writing SQL commands.**

Database Design and Software Engineering

Developing a database application involves two related activities:

- Database Design
- Software Engineering

Database design focuses on designing tables, relationships, and constraints.

Software engineering focuses on developing the application interface, reports, and program logic.

ER Model

The **Entity–Relationship (ER) Model** is a **high-level conceptual data model** used to design databases.

It describes:

- **Entities**
- **Attributes**
- **Relationships**

before implementing the database.

ER Diagram

- An ER Diagram (ERD) is a graphical representation of the ER model.
- It shows entities, attributes, relationships, and constraints visually.

UML (Unified Modeling Language)

Unlike ER diagrams, UML class diagrams include both data (attributes) and operations (methods).

Why is the ER Model Important?

The ER Model helps designers:

- **Understand** user requirements.
- **Identify** entities and relationships.
- **Create** a database before implementation.
- **Reduce** design errors.
- **Communicate** effectively with users.

3.1 Using High-Level Conceptual Data Models for Database

- Database design is a step-by-step process used **to create a database that satisfies users' requirements.**
- **Before creating tables,** designers must first understand **what users need and then design the database at different levels.**

DBMS-independent (Above the dashed line)

These activities are **independent of any specific DBMS** (such as MySQL, Oracle, SQL Server).

At this stage, designers focus only on understanding the business requirements and designing the database conceptually.

In the figure, the DBMS-independent activities are:

- **Miniworld**
- **Requirements Collection and Analysis**
- **Functional Analysis**
- **Data Requirements**
- **Conceptual Design**
- **Conceptual Schema (in a high-level data model)**
- **High-Level Transaction Specification**

These steps do **not** require SQL or a particular DBMS.

DBMS-independent

- Requirements Collection and Analysis
- Functional Analysis
- Conceptual Design
- Conceptual Schema

DBMS-specific

- Logical Design
- Physical Design
- Application Program Design
- Transaction Implementation

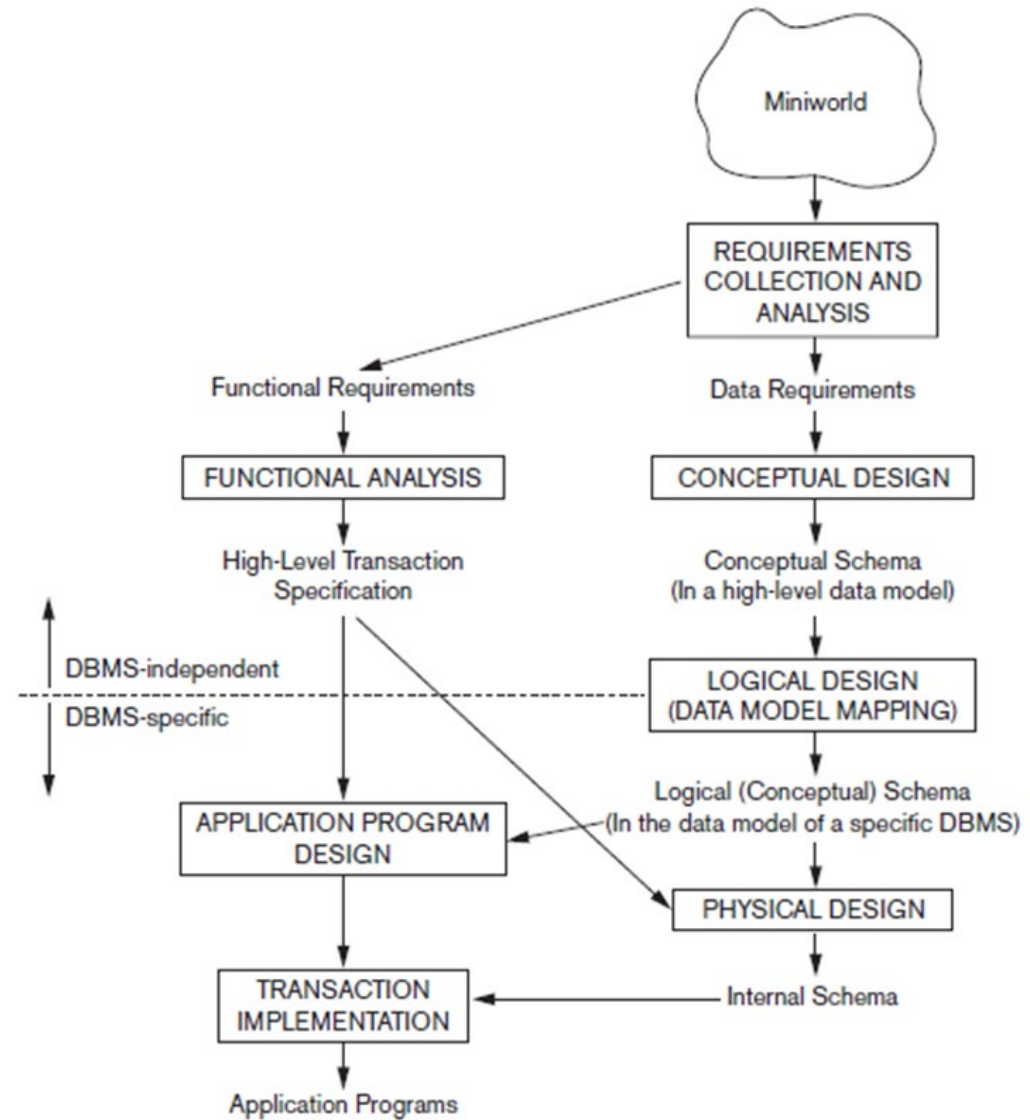


Figure 3.1

A simplified diagram to illustrate the main phases of database design.

1. Requirements Collection and Analysis

The first step is to collect and analyze the users' requirements.

Database designers interview users to understand:

- What information should be stored?
- What reports are needed?
- What operations will users perform?

The result is a complete list of user requirements.

Example: **Library System**

User Requirements

- Store book information
- Store member information
- Borrow books
- Return books
- Print reports

Functional Requirements

Besides data requirements, designers identify **the functional requirements**.

Functional requirements **describe what operations users perform on the database**.

Examples include **Insert, Update, Delete, Search and Print etc.**

Functional Analysis

Functional analysis studies the required operations and produces **High-Level Transaction Specifications**.

Data Requirements

Data requirements describe **what data must be stored**.

2. Conceptual Design

After collecting requirements, designers create a **conceptual schema** using a high-level conceptual data model such as the **ER Model**.

The conceptual schema describes:

- Entities
- Attributes
- Relationships
- Constraints

No implementation details are included.

Example

Student

- StudentID
- Name
- Major

Course

- CourseID
- Title

Relationship: Student **Enrolls** in Course.

High-Level Transaction Specification

This specifies the database operations that users will perform.

Examples:

- Register Student
- Pay Hostel Fee
- Assign Room

Why Use a High-Level Conceptual Model?

Advantages:

- Easy to understand
- Easy to communicate with users
- Independent of DBMS
- Focuses on business requirements

Conceptual Schema

A conceptual schema is a DBMS-independent description of the database.

It does not contain SQL or implementation details.

Logical Design (Data Model Mapping)

The conceptual schema is converted into the data model supported by the DBMS.

Usually, it is transformed into relational tables.

Physical Design

Physical design specifies how the database is stored.

It includes

- Files
- Indexes
- Storage
- Access Paths

Application Program Design

Application programmers develop the software that interacts with the database.

Examples:

- Login Form
- Registration Form
- Search Page
- Reports

Transaction Implementation

Finally, all transactions are implemented using SQL and programming languages.

Examples:

- INSERT
- UPDATE
- DELETE
- SELECT

3.2 A Sample Database Application (COMPANY Database)

- This section introduces a sample database called **COMPANY** to explain the **Entity-Relationship (ER) Model**.
- The **COMPANY** database stores information about **employees, departments, and projects**.

Requirement 1: Department

The company is organized into **departments**.

Each department has:

- A **unique department name**
- A **unique department number**
- One employee who manages the department
- The date when the manager started managing
- One or more department locations

Requirement 2: Project

Each department controls several projects.

Each project has:

- A unique project name
- A unique project number
- One project location

Requirement 3: Employee

The database stores:

- Employee Name
- Social Security Number (SSN)
- Address
- Salary
- Gender
- Birth Date

- ❖ Each employee belongs to one department.
- ❖ An employee may work on many projects.
- ❖ The database stores the number of hours worked per week on each project.
- ❖ Each employee has one direct supervisor.

Requirement 4: Dependent

The company stores information about employee dependents for insurance purposes.

Each dependent has:

- First Name
- Gender
- Birth Date
- Relationship

Relationship	Meaning
Department → Employee	1 : N (One department has many employees.)
Department → Project	1 : N (One department controls many projects.)
Employee → Project	M : N (Many employees work on many projects.)
Employee → Dependent	1 : N (One employee has many dependents.)
Employee → Employee	1 : N (One supervisor supervises many employees.)

3.3 Entity Types, Entity Sets, Attributes, and Keys

The **Entity-Relationship (ER) Model** represents data using three basic concepts:

1.Entity

2.Relationship

3.Attribute

Employee

Attribute	Value
Name	John Smith
Address	Houston
Age	55
Phone	713-749-2630

3.3.1 Entities and Attributes

1. What is an Entity?

An **Entity** is a real-world object or thing that exists independently.

It can be **Physical object** or **Conceptual object**.

Physical Entity Examples: Student, Teacher, Employee, Car, House

Conceptual Entity Examples: Company, Course, Department, Job, Project

Entity

It represents a real-world object, concept or thing about which data is stored in a database.

It act as a building block of a database.

Tables in relational database represent these entities.

Example of entities:

- **Real-World Objects:** Person, Car, Employee etc.
- **Concepts:** Course, Event, Reservation etc.
- **Things:** Product, Document, Device etc.

The entity type defines the structure of an entity, while individual instances of that type represent specific entities.

Types of Entity

There are two main types of entities:

1. Strong Entity

A type of entity that has a key Attribute that can uniquely identify each instance of the entity.

A Strong Entity **does not depend on any other Entity in the Schema** for its identification.

It has a **primary key** that ensures its uniqueness and is represented **by a rectangle in an ER diagram.**

2. Weak Entity

It **cannot be uniquely identified** by its own attributes alone.

It depends on **a strong entity to be identified.**

A weak entity is associated with an identifying entity (strong entity), which helps in its identification.

A weak entity **are represented by a double rectangle.**

The **participation** of weak entity types is always **total.**

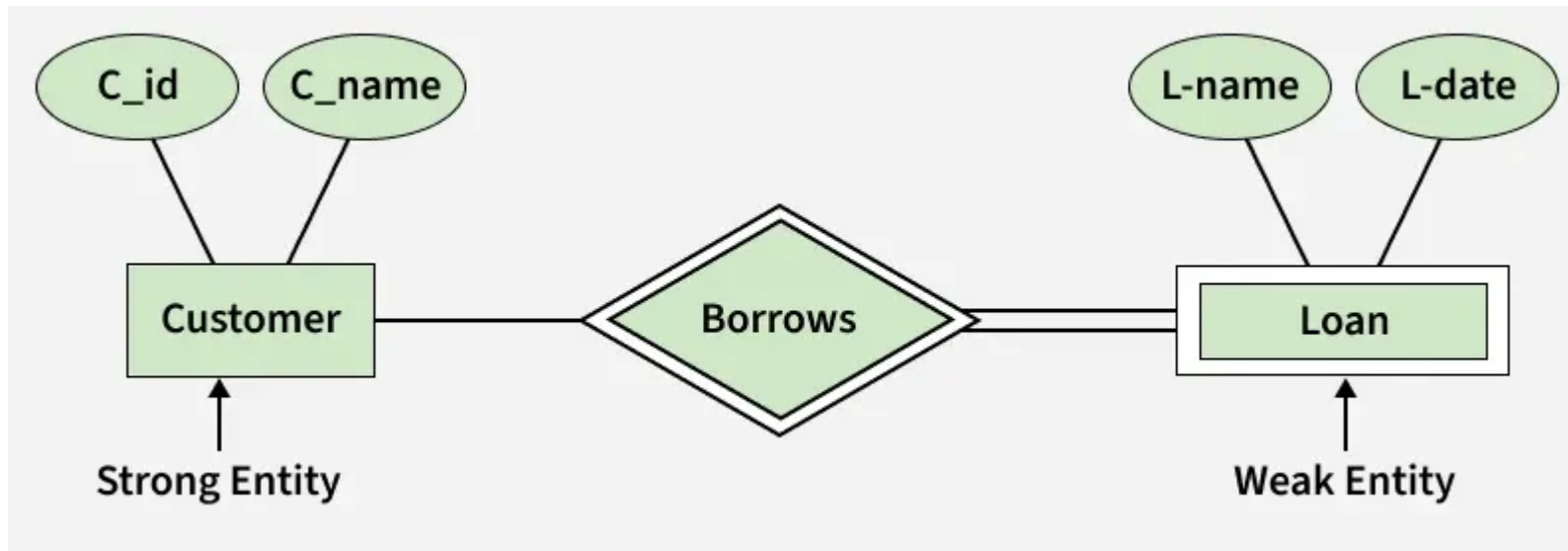
The relationship between the weak entity type and its identifying strong entity type is called **identifying relationship** and it is represented by a double diamond.

Example:

A **company** may store the information of **dependents (Parents, Children, Spouse) of an Employee**.

But the dependents **can't exist without the employee**.

So dependent will be a Weak Entity Type and Employee will be identifying entity type for dependent, **which means it is Strong Entity Type**.

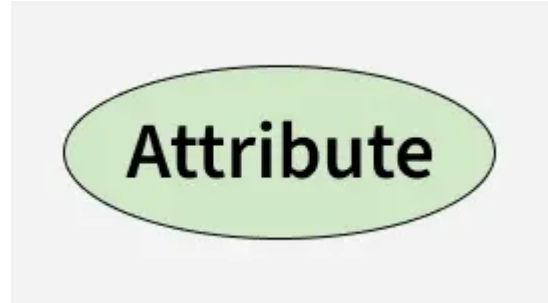


Attributes in ER Model

Attributes are the properties that define the entity type.

For example, for a **Student entity** Roll_No, Name, DOB, Age, Address, and Mobile_No are the attributes that define entity type Student.

In ER diagram, the attribute is **represented by an oval**.



Types of Attributes

1. Key Attribute

The attribute which **uniquely identifies each entity in the entity set** is called the **key attribute**.

For example, Roll_No will be unique for each student.

In ER diagram, the key attribute is **represented by an oval with an underline**.

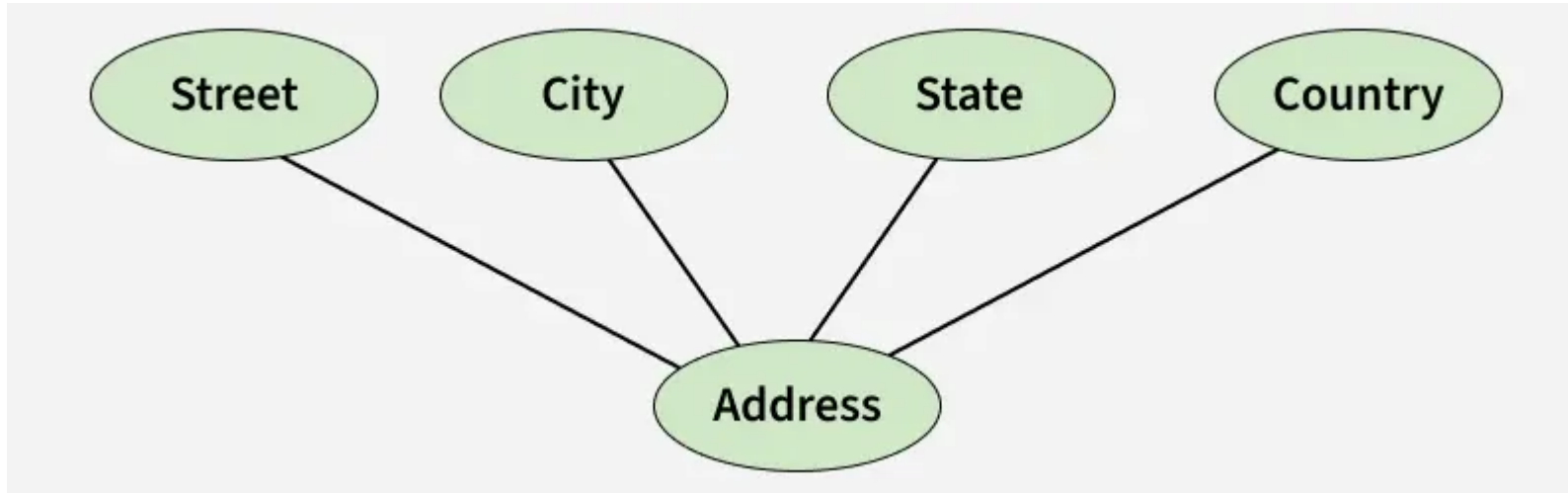


2. Composite Attribute

An attribute composed of many other attributes is called a composite attribute.

For example, the Address attribute of the student Entity type consists of **Street, City, State, and Country**.

In ER diagram, the composite attribute is **represented by an oval comprising of ovals**.



Multivalued Attribute

An attribute consisting of **more than one value for a given entity**.

For example, **Phone_No** (can be more than one for a given student).

In ER diagram, a multivalued attribute is **represented by a double oval**.

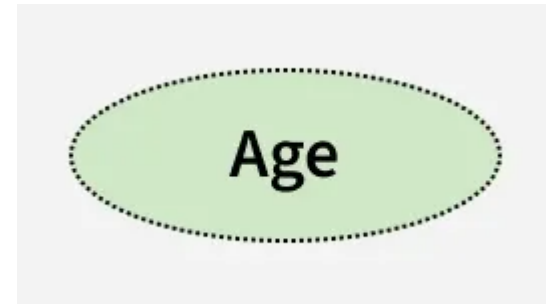


Derived Attribute

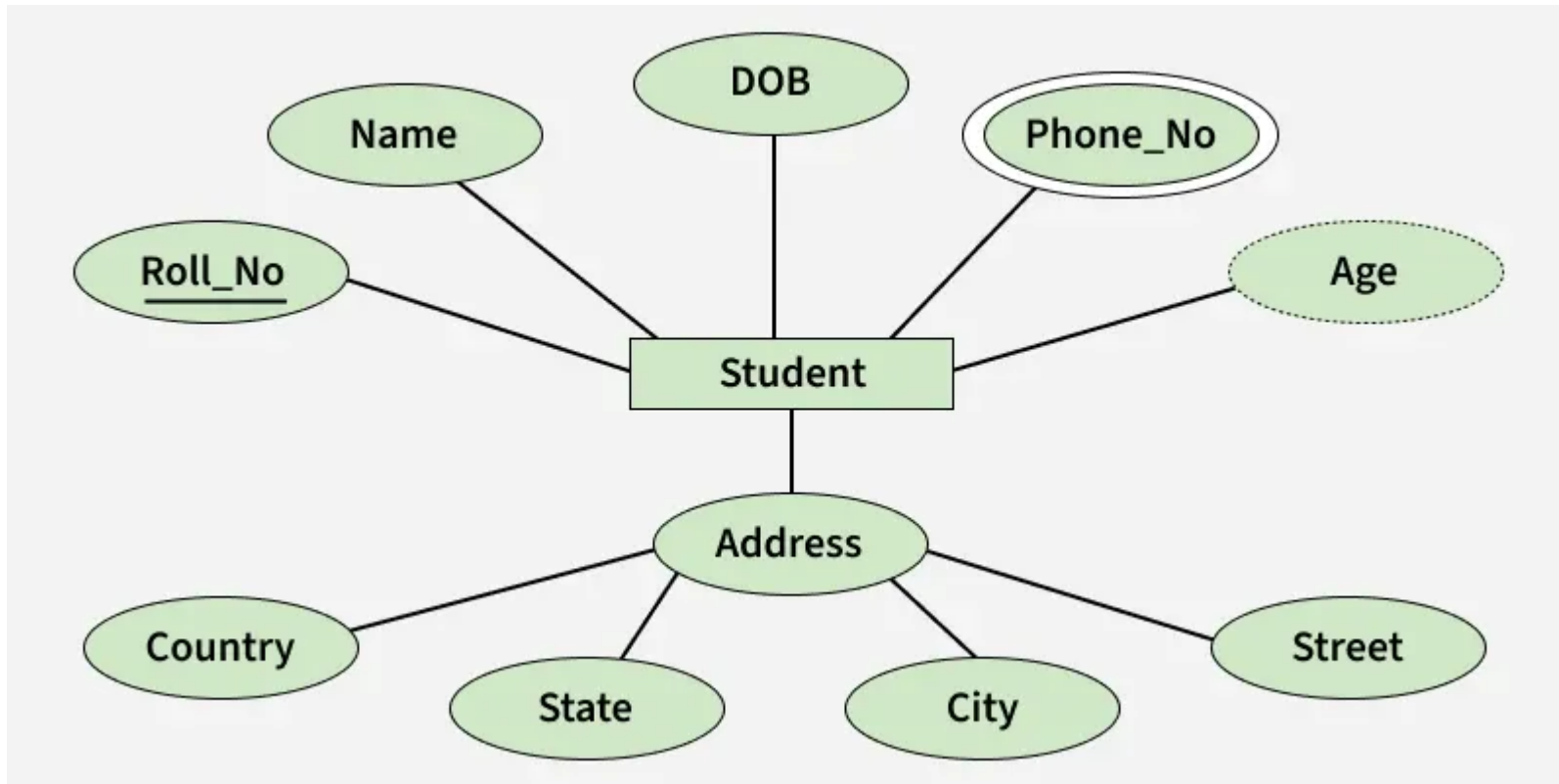
An attribute that **can be derived from other attributes of the entity type** is known as a derived attribute.

e.g.; Age (can be derived from DOB).

In ER diagram, the derived attribute is **represented by a dashed oval**.



The **Complete Entity Type Student** with its **Attributes** can be represented as:



Attribute

An **Attribute** is a property that describes an entity.

Types of Attributes

1. Simple Attribute (Atomic Attribute)

Cannot be divided into smaller parts.

Example Attribute: Age, Gender, Salary, StudentID

2. Composite Attribute

Can be divided into smaller meaningful parts.

3. Single-Valued Attribute

Only one value for each entity.

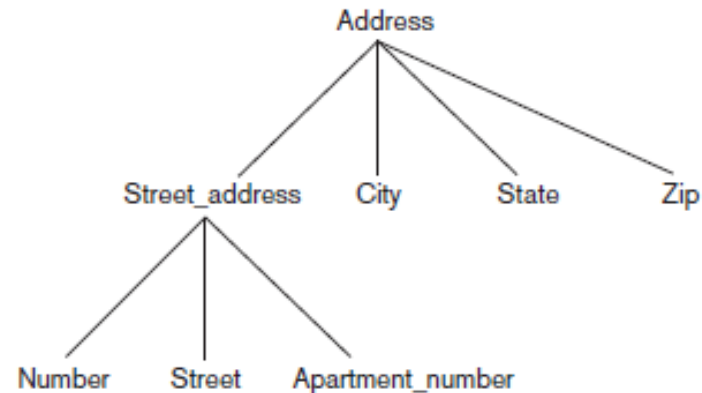
Example:

StudentID = 202301

Gender = Female

DOB = 12-05-2003

Simple	Composite
Cannot be divided	Can be divided into smaller parts
Example: Age	Example: Address



4. Multivalued Attribute

Can have multiple values for one entity.

5. Stored Attribute

Stored directly inside the database.

6. Derived Attribute

Calculated from another attribute.

Example: Department -----> Employees -----> Number_of_Employees

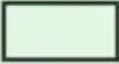
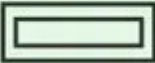
Single-valued	Multivalued
One value	Multiple values
Gender	Phone Numbers

Stored vs Derived Attributes

Stored	Derived
Birth Date	Age
Salary	Total Salary
Price	Discount Price
Marks	Grade

Symbols Used in ER Model

ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

7. NULL Value

NULL means **no value**.

It does **NOT** always mean **zero**.

8. Complex Attribute

Combination of Composite and Multivalued attributes.

{Address_phone ({Phone(Area_code,Phone_number)}, Address(Street_address(Number,Street,Apartment_number),
City,State,Zip))}

3.3.2 Entity Types, Entity Sets, Keys, and Value Sets

1. Entity Type

An **Entity Type** is a collection (class) of entities that have the **same attributes**.

It describes the **structure (schema)** of similar entities.

Entity Type -----> **Template (Schema)**

Entity -----> **Actual Data (Instance)**

2. Entity Set

An **Entity Set** is the collection of all entities belonging to the same entity type at a particular time.

Entity Set

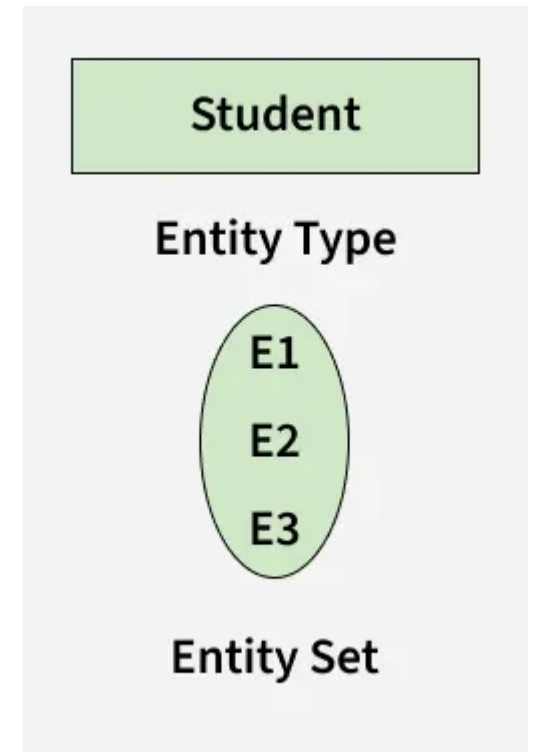
Refers to an individual object of an entity type, and **the collection of all entities of a particular type is called an entity set.**

For example, **E1 is an entity that belongs to the entity type "Student," and the group of all students forms the entity set.**

We can represent **the entity sets in an ER Diagram**

but we can't represent **individual entities** because **an entity is like a row in a table,** and an **ER diagram shows the structure and relationships of data,** not specific data entries (like rows and columns).

An ER diagram is a visual representation of the data model, not the actual data itself.

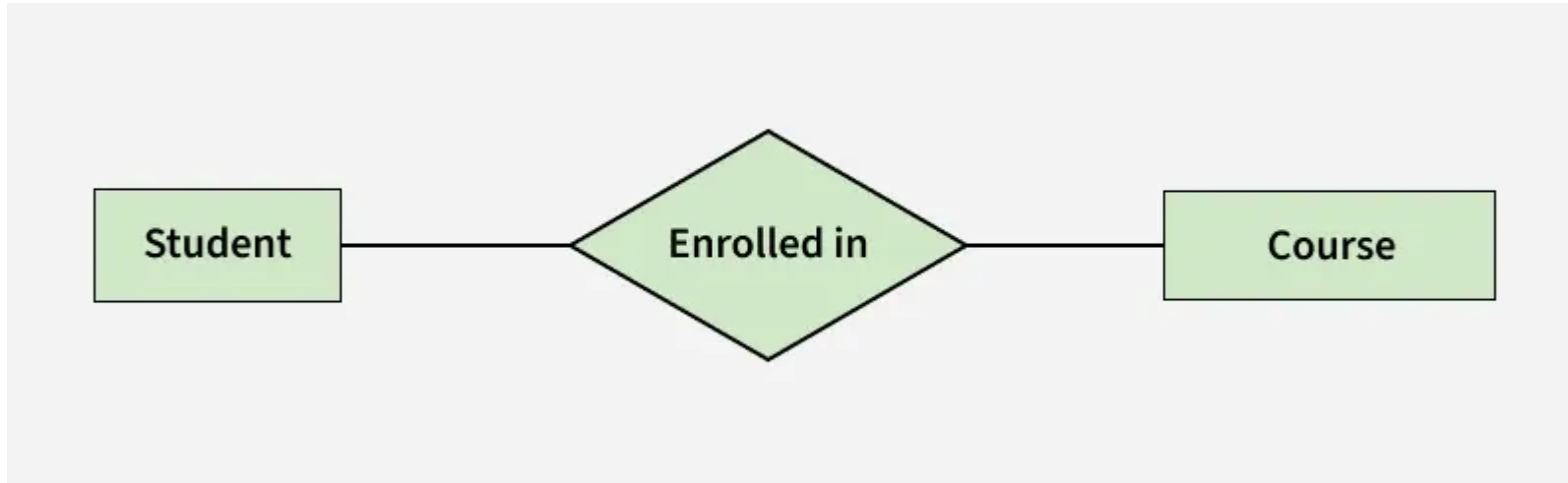


Relationship Type and Relationship Set

A Relationship Type represents the association between entity types.

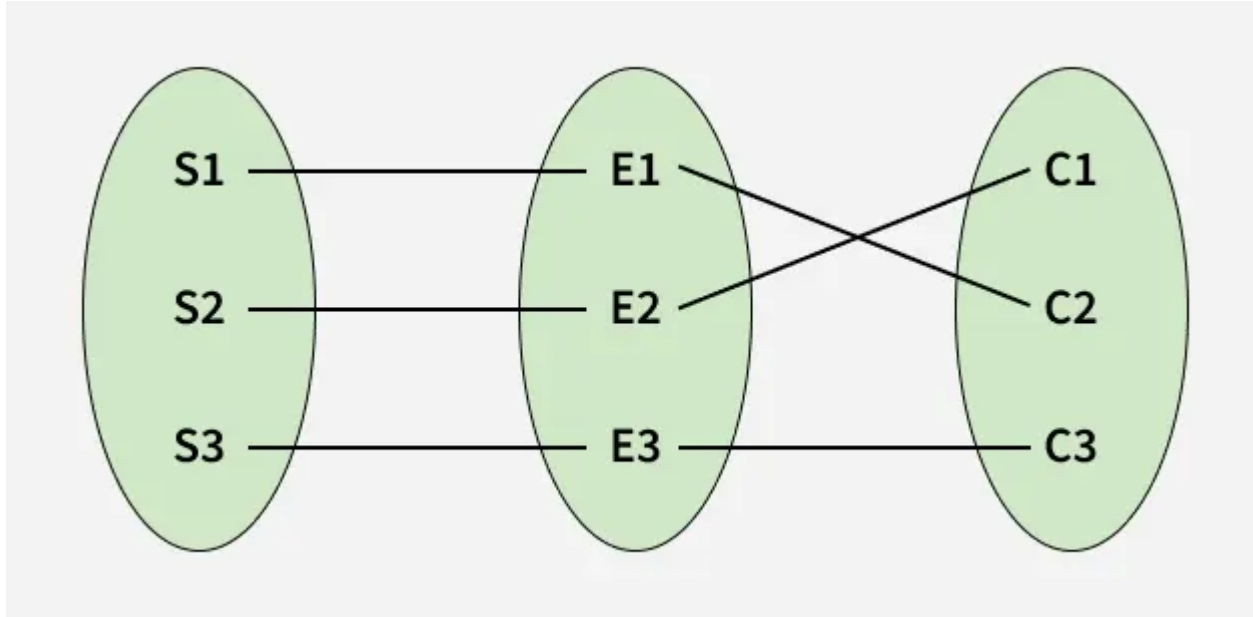
For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course.

In ER diagram, the relationship type is **represented by a diamond and connecting the entities with lines.**



A set of relationships of the same type is known as a relationship set.

The following relationship set depicts S1 as enrolled in C2, S2 as enrolled in C1, and S3 as registered in C3.

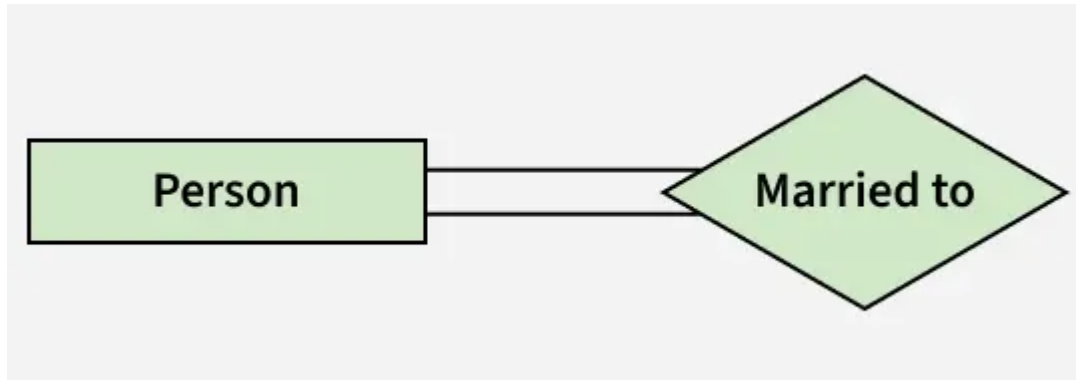


Degree of a Relationship Set

The number of different entity sets participating in a relationship set is called the degree of a relationship set.

1. Unary/Recursive Relationship: When there is **only ONE** entity set participating in a relation.

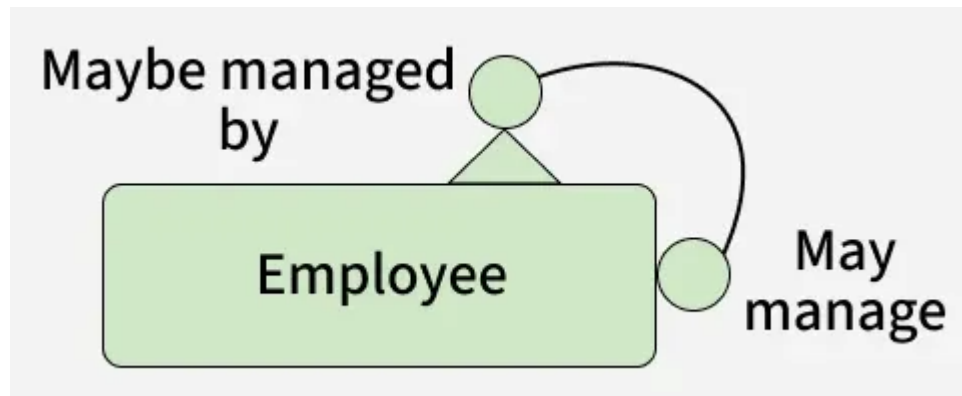
For example, one person is married to only one person.



A relationship between two entities of the same entity set is called a recursive relationship or repeated relationship.

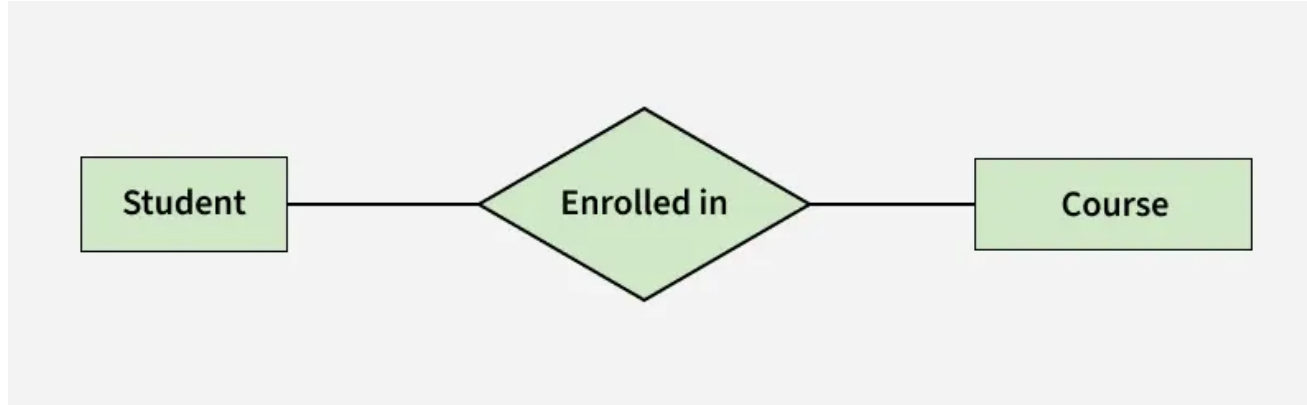
Here, the same entity set participates more than once in a relationship type with a different role for each instance.

- To represent a recursive relationship in an ER diagram, we use a self-join, **which is a join between a table and itself.**
- The self-join involves **creating two instances of the same entity and connecting them with a relationship, one is considered as the child and the other as the parent.**
- Often used **to represent hierarchies or networks**, where an entity can be connected to other entities of the same type.

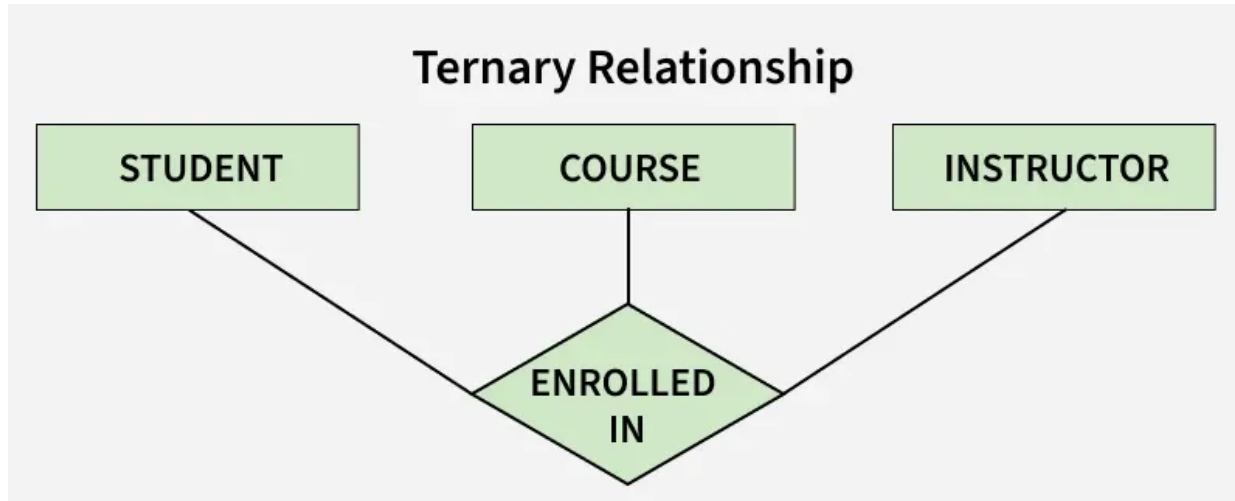


Binary Relationship: When there are TWO entities set participating in a relationship.

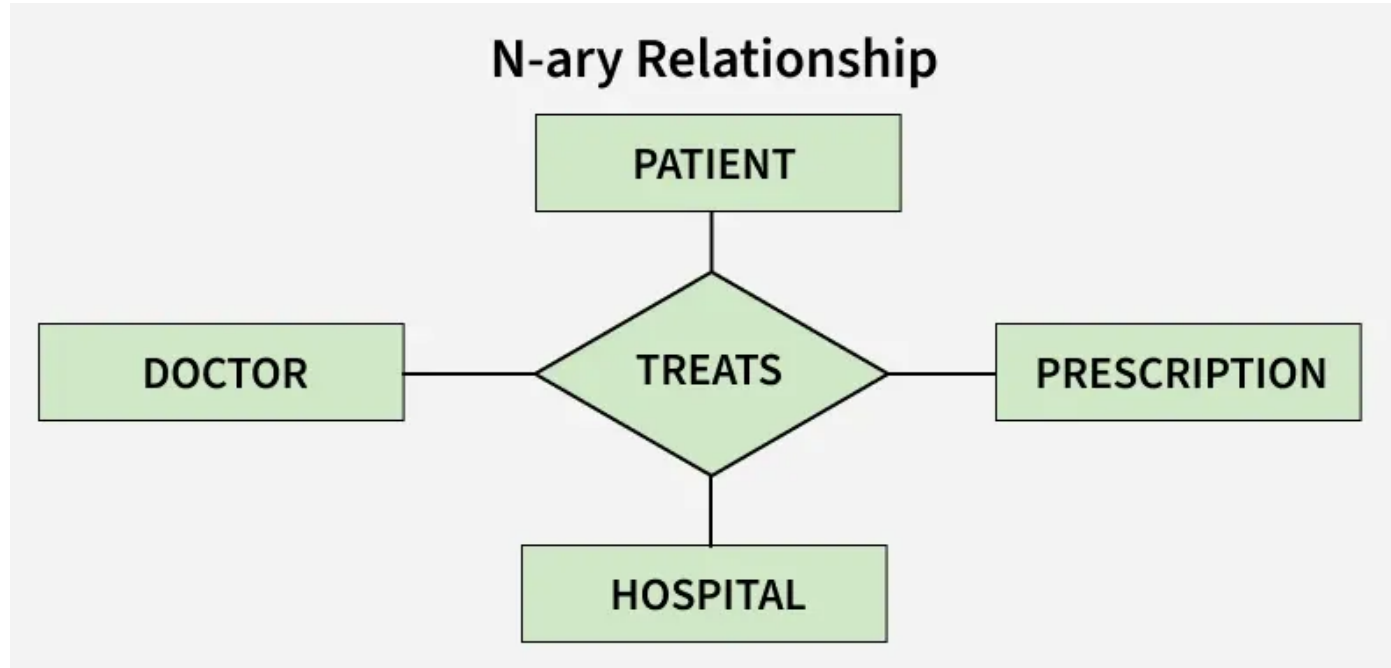
For example, a Student is enrolled in a Course.



Ternary Relationship: When there are three entity sets participating in a relationship.



N-ary Relationship: When there are n entities set participating in a relationship, the relationship is called an n-ary relationship.



Cardinality in ER Model

The maximum number of times an entity of an entity set participates in a relationship set is known as cardinality.

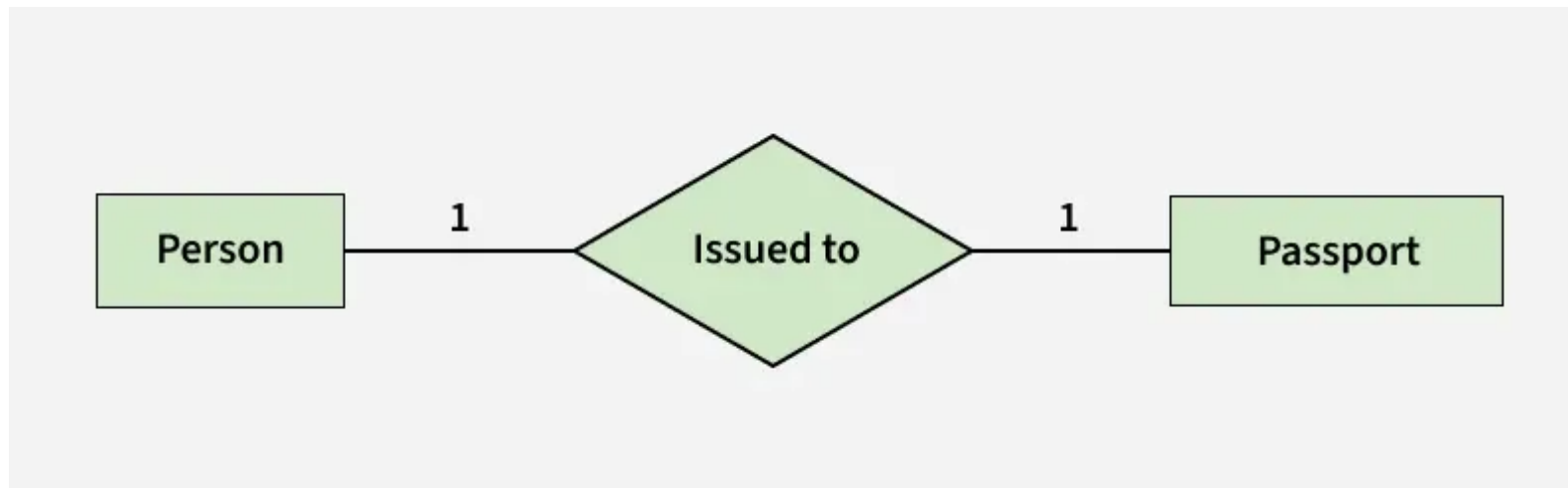
Cardinality can be of different types:

1. One-to-One

When **each entity in each entity set can take part only once in the relationship**, the cardinality is **one-to-one**.

Let us assume that **one person can be issued only one passport**, and **one passport is issued to only one person**.

So, the relationship will be One-to-One (1 : 1), meaning that **each person has a single passport, and each passport belongs to a single person**.

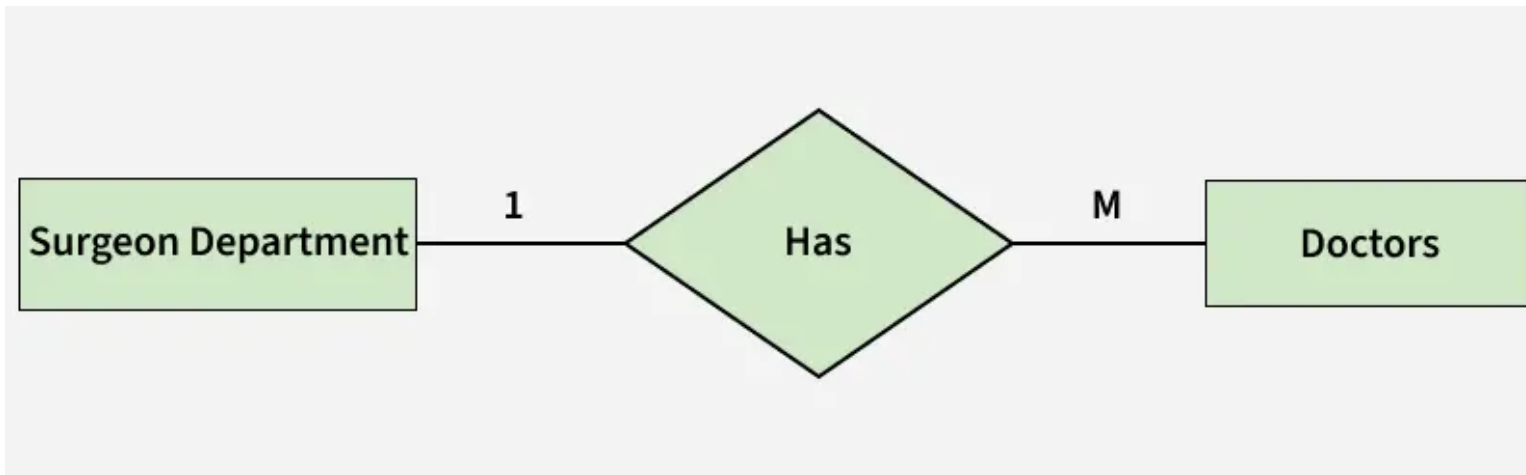


2. One-to-Many

In a one-to-many relationship, **one entity can be associated with multiple entities.**

For example, **a single Surgeon Department can have many Doctors.**

Therefore, the cardinality of this relationship is 1 to M, **meaning one department can have many doctors.**



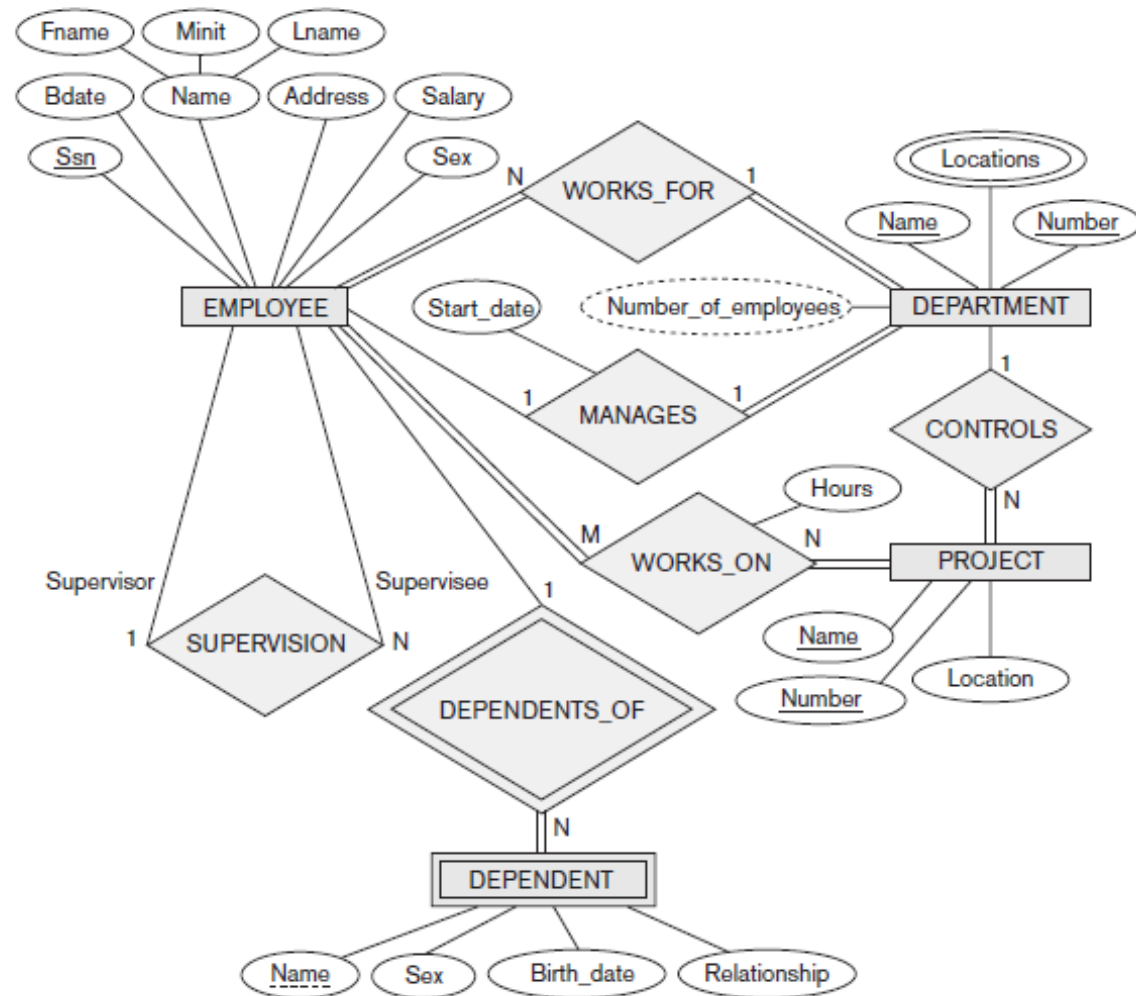


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 3.14.